

TECHNICAL OVERVIEW

ShopZone

A full-stack e-commerce platform built with Spring Boot 3 and React 18 — architecture, data model, security, and complete use-case reference.

Spring Boot 3.3

Java 17

React 18 + Vite 5

PostgreSQL

Spring Security + JWT

REST API

Recharts

§ Contents

- 1 Application at a Glance (the one-paragraph overview)
- 2 Technology Stack
- 3 High-Level System Architecture
- 4 Backend Layered Architecture
- 5 Data Model (Entity-Relationship Diagram)
- 6 Authentication & Request Security Flow
- 7 Checkout / Order Sequence Flow
- 8 Order Status State Machine
- 9 Complete Use-Case Reference
- 10 REST API Endpoint Catalogue
- 11 Frontend Structure & Routing
- 12 Running the Project & Seed Data

1. Application at a Glance

ShopZone is a full-stack e-commerce store where shoppers can browse a catalogue of products, filter and search them, read and leave star reviews, add items to a persistent shopping cart, save shipping addresses, and place orders — while store administrators manage the catalogue and fulfil orders from a dedicated dashboard. The **backend** is a Spring Boot 3 REST API (Java 17) organised into clean controller → service → repository → entity layers, persisting to PostgreSQL through Spring Data JPA / Hibernate, and secured with stateless JWT authentication and role-based authorisation (**USER** vs **ADMIN**). The **frontend** is a React 18 single-page app (built with Vite) that talks to the API through an Axios service layer, keeps a JWT in **localStorage** , holds global auth and cart state in React Context, and guards private routes behind login and admin checks. When an order is placed the API runs the whole thing as one transaction: it re-checks stock, decrements inventory, snapshots each line-item's price, adds a flat shipping fee, saves the order in **PENDING** state, and empties the cart — after which an admin walks the order through **CONFIRMED** → **SHIPPED** → **DELIVERED** (or **CANCELLED**). In short, it is a compact but realistic reference implementation of a store: catalogue, cart, checkout, orders, reviews, and an admin back-office, all wired end to end.

What a customer can do

- Register / log in (JWT)
- Browse, filter, sort & search products
- View product detail, gallery & reviews
- Manage a persistent cart
- Save & reuse shipping addresses
- Check out and place orders
- View order history & detail
- Leave one star-rating + review per product

What an admin can do

- See a dashboard: products, orders, users, revenue
- View a bar chart of orders by status
- Create / edit products (with image URLs)
- Soft-delete products (deactivate)
- Create / edit categories
- View *all* orders across users
- Advance an order's status
- View any single order's full detail

2. Technology Stack

Backend

Concern	Choice
Language / Runtime	Java 17
Framework	Spring Boot 3.3
Web	Spring MVC (REST)
Security	Spring Security + JWT (jjwt 0.12)
Persistence	Spring Data JPA + Hibernate
Database	PostgreSQL
Password hashing	BCrypt
Build	Maven

Frontend

Concern	Choice
Library	React 18
Build tool / dev server	Vite 5
Routing	React Router v6
HTTP client	Axios (+ interceptors)
Forms / validation	React Hook Form + Yup
Charts	Recharts
State	React Context (Auth, Cart)
Styling	Plain CSS Modules

How the two halves connect

The React dev server runs on `:5173` and proxies `/api` calls to the Spring Boot server on `:8080` (configured in `vite.config.js`). The backend's CORS config explicitly allows the `:5173` origin with credentials. Every authenticated request carries an `Authorization: Bearer <JWT>` header injected by the Axios request interceptor.

3. High-Level System Architecture

Three tiers: a React SPA in the browser, a stateless Spring Boot REST API, and a PostgreSQL database. The client never touches the database directly — all access flows through the API, which authenticates every protected request via a JWT filter before it reaches a controller.

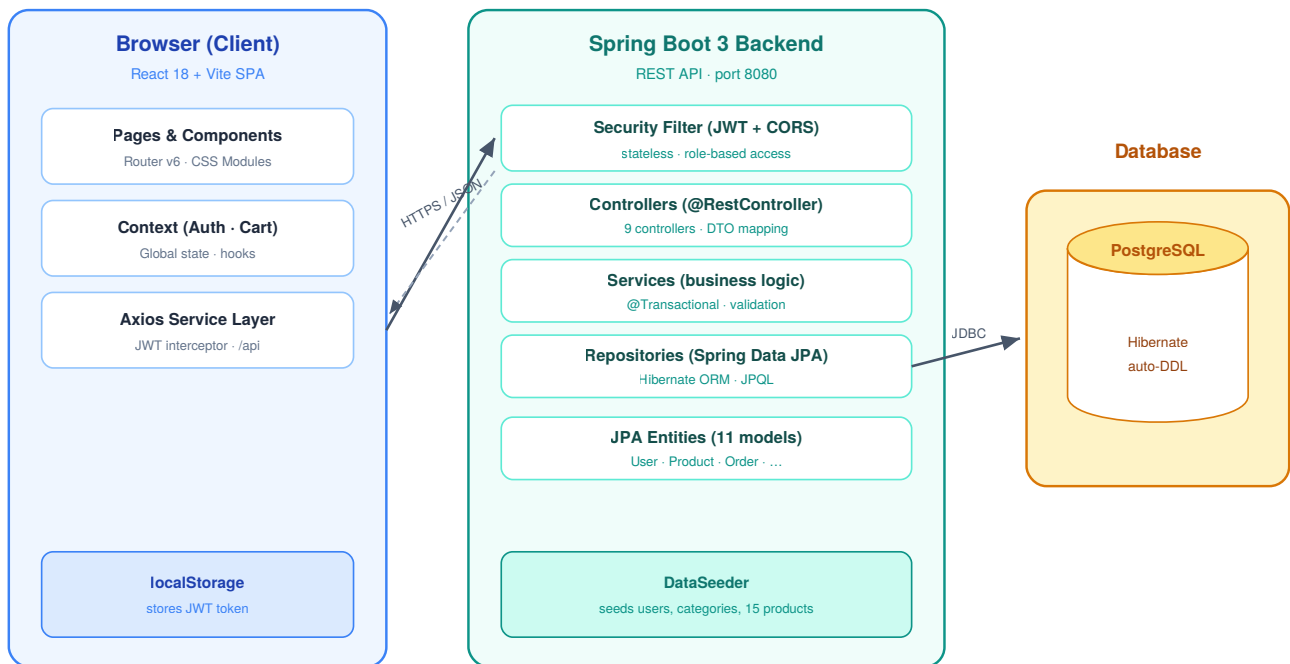


Figure 3.1 — Three-tier architecture and the path of a request from browser to database.

4. Backend Layered Architecture

The backend follows the classic Spring layering. A request enters through the **security filter**, reaches a **controller** (thin — it only maps HTTP and delegates), which calls a **service** holding the transactional business rules, which in turn uses **repositories** (Spring Data JPA) to read and write **entities**. Errors thrown anywhere are translated into clean HTTP responses by a global exception handler.

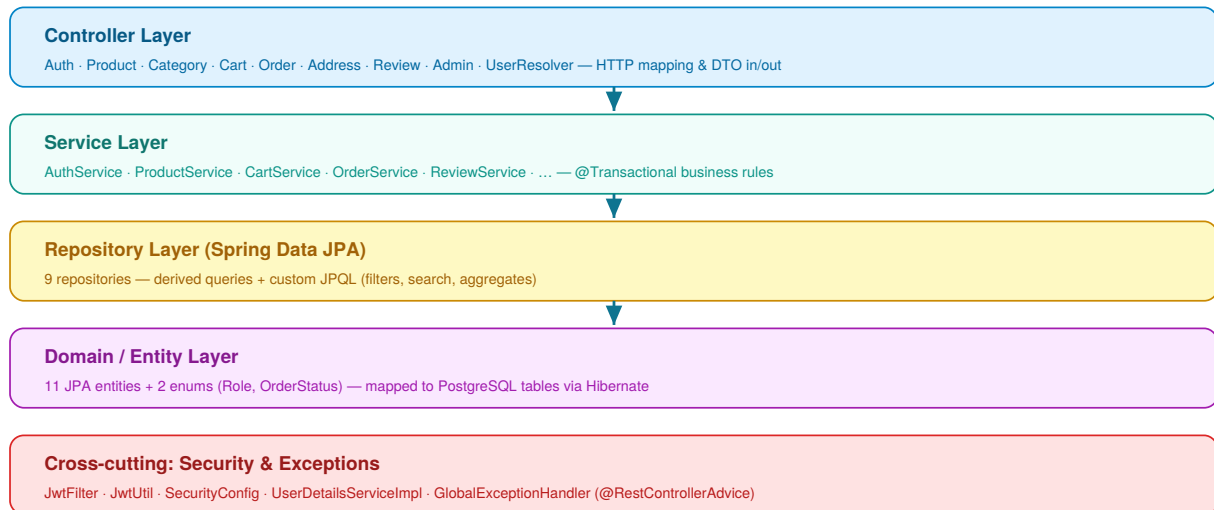


Figure 4.1 — Layered backend architecture with cross-cutting security and error handling.

Package layout

Package	Responsibility	Notable classes
<code>controller</code>	HTTP endpoints, DTO in/out	Auth, Product, Cart, Order, Category, Address, Review, Admin, UserResolver
<code>service</code>	Business logic, transactions	AuthService, ProductService, CartService, OrderService, ReviewService, ...
<code>repository</code>	Data access (JPA)	9 <code>JpaRepository</code> interfaces with custom JPQL
<code>model</code>	JPA entities + enums	User, Product, Order, OrderItem, Cart... + Role, OrderStatus
<code>dto</code>	Request / response shapes	*Request, *Response, DashboardStatsDto
<code>security</code>	JWT + auth wiring	JwtFilter, JwtUtil, SecurityConfig, UserDetailsServiceImpl
<code>exception</code>	Error handling	GlobalExceptionHandler + custom exceptions

5. Data Model (Entity-Relationship Diagram)

Eleven entities model the domain. A **User** owns addresses, cart items, orders and reviews. A **Product** belongs to a (self-nesting) **Category** and has images and reviews. An **Order** is composed of **OrderItems** that snapshot each product's price at purchase time, so later price changes never rewrite order history.

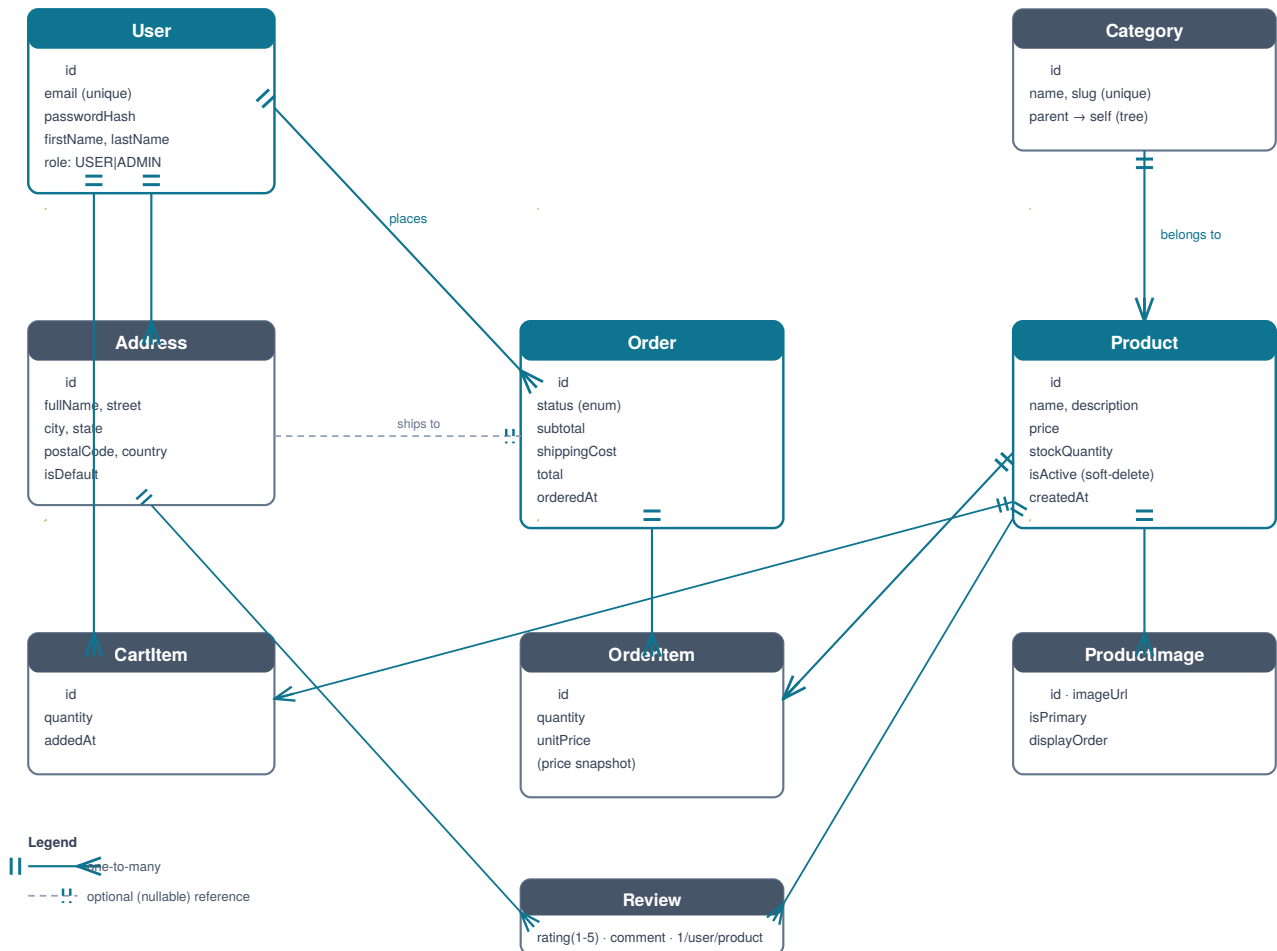


Figure 5.1 — Entity-relationship diagram. Teal boxes are the core aggregates; grey boxes are dependent entities.

Design points worth noting

- **Price snapshotting:** `OrderItem.unitPrice` is copied from the product at checkout, decoupling historical orders from live prices.
- **Soft delete:** products are never physically removed — `isActive=false` hides them from all public queries.
- **Self-referencing categories:** `Category.parent` allows a tree (parent/children) exposed as a nested structure to the client.
- **One review per user per product:** enforced in the service via `existsByUserIdAndProductId`.
- **Default address:** setting one address default clears the flag on the user's other addresses in the same transaction.

6. Authentication & Request Security Flow

Authentication is **stateless**: there are no server sessions. On login the server verifies the BCrypt password hash and returns a signed JWT (HS256, 24-hour expiry). The client stores it and sends it on every subsequent request; a `OncePerRequestFilter` validates the token and populates the Spring Security context with the user's role before the request reaches a controller.

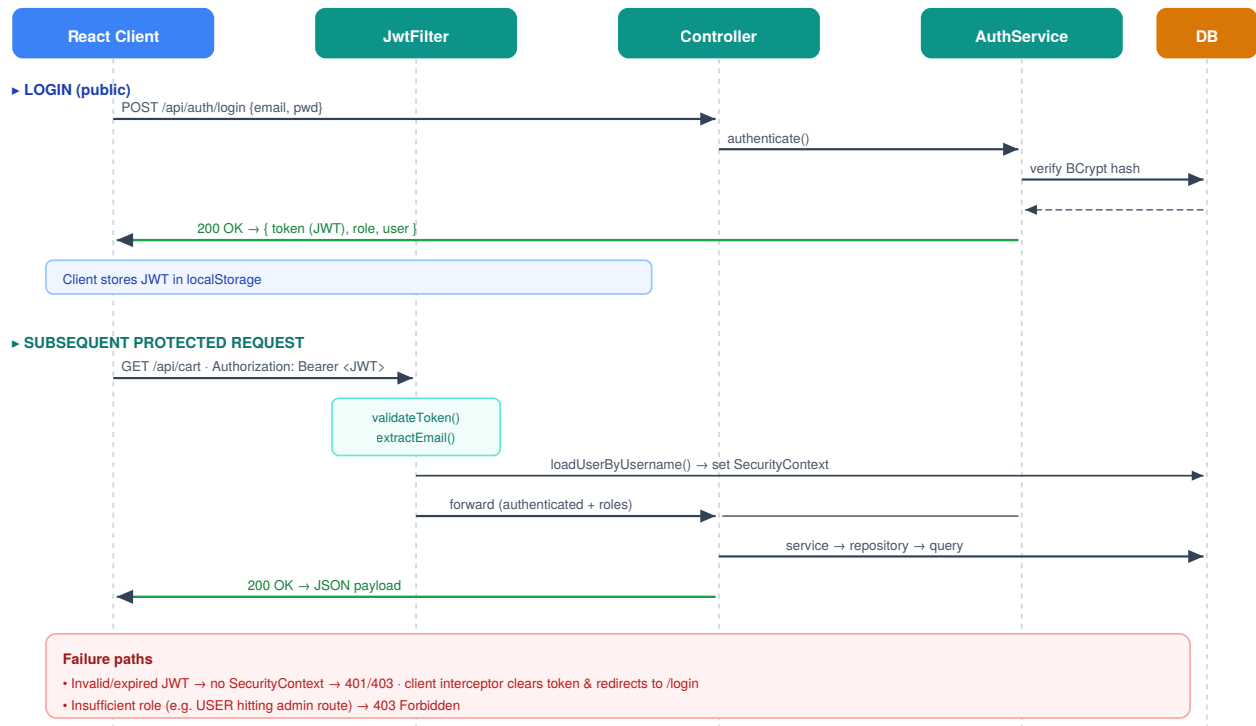


Figure 6.1 — Login and the lifecycle of an authenticated request, including failure paths.

Authorisation rules (from `SecurityConfig`)

Rule	Access
<code>/api/auth/**</code>	PUBLIC
<code>GET /api/products/**</code> , <code>GET /api/categories/**</code>	PUBLIC
<code>POST/PUT/DELETE /api/products/**</code> , <code>POST/PUT /api/categories/**</code>	ADMIN
<code>/api/admin/**</code> , <code>PATCH /api/orders/*/status</code> , <code>GET /api/orders/all</code>	ADMIN
Everything else (cart, own orders, addresses, posting reviews)	AUTHENTICATED

Client-side enforcement mirrors the server

In React, `<ProtectedRoute>` redirects unauthenticated users to `/login`, and `<AdminRoute>` additionally bounces non-admins to `/`. This is convenience only — the real gate is the server's role checks, since anyone can call the API directly.

7. Checkout / Order Sequence Flow

Placing an order is the most involved operation and is wrapped in a single `@Transactional` method, so either every step succeeds or the whole thing rolls back. The sequence below shows the guards (empty cart, insufficient stock, address ownership) and the money math.

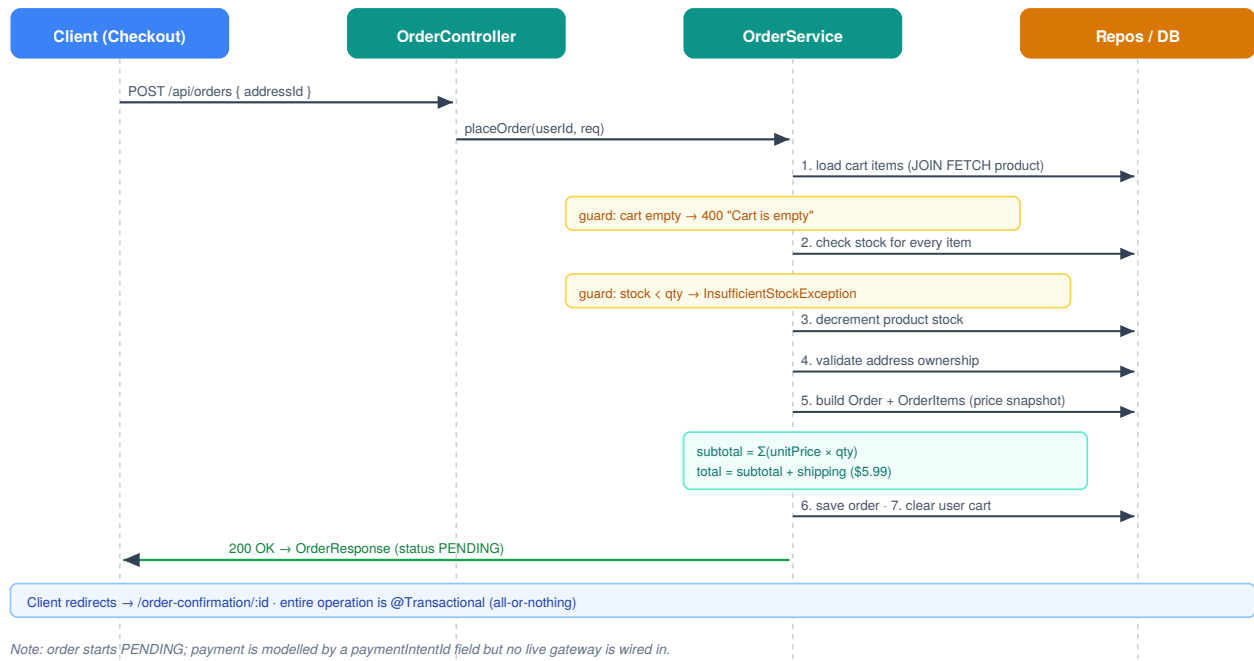


Figure 7.1 — `placeOrder` sequence: validate → decrement stock → snapshot prices → save → clear cart.

8. Order Status State Machine

Every order carries an `OrderStatus` enum. Orders are born `PENDING` at checkout, and an admin advances them. Cancelled orders are excluded from the revenue calculation on the admin dashboard.

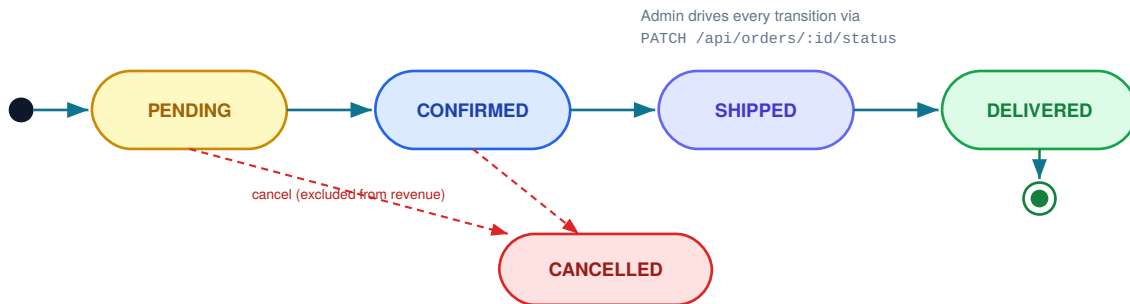


Figure 8.1 — Order status lifecycle. Transitions are performed by admins via the status endpoint.

Implementation note

The API's `updateStatus` accepts any valid enum value (it does not currently enforce a strict forward-only progression), so an admin has full manual control over an order's status. Invalid values return a 400.

9. Complete Use-Case Reference

This section enumerates every use case the application supports, grouped by actor. Each entry lists the actor, the trigger, the main flow, the key business rules enforced, and the primary endpoint(s) involved — all derived directly from the source code.

A. Account & Authentication

UC-1 — Register a new account

Actor	Guest (unauthenticated visitor)
Flow	Visitor submits email, password, first & last name → server rejects duplicate emails → password is BCrypt-hashed → user is created with role <code>USER</code> → a JWT is returned and the visitor is logged in immediately.
Rules	Email must be unique; new users are always <code>USER</code> (never self-assign admin).
Endpoint	<code>POST /api/auth/register</code> PUBLIC

UC-2 — Log in

Actor	Registered user or admin
Flow	Submit email + password → credentials verified against the BCrypt hash → JWT returned with role and profile → client stores token in <code>localStorage</code> .
Rules	Wrong credentials → authentication failure (401).
Endpoint	<code>POST /api/auth/login</code> PUBLIC

UC-3 — Restore session / fetch current user

Actor	Authenticated user
Flow	On app load, if a token exists the client calls <code>/me</code> to rehydrate the user object (role, name, email). A failure clears the stale token.
Endpoint	<code>GET /api/auth/me</code> AUTH

UC-4 — Log out

Actor	Authenticated user
Flow	Client-side only: token removed from <code>localStorage</code> , auth/cart state cleared. (Stateless JWT — nothing to invalidate server-side.)

B. Browsing the Catalogue

UC-5 — Browse products with filters & sorting

Actor	Anyone (public)
Flow	Paginated product list; filter by category, min/max price, and in-stock; sort by newest (default), price ascending, or price descending. Only active products appear.
Rules	Soft-deleted (<code>isActive=false</code>) products are excluded from every listing.
Endpoint	<code>GET /api/products?page&size&sort&categoryId&minPrice&maxPrice&inStock</code> PUBLIC

UC-6 — Search products

Actor	Anyone (public)
Flow	Free-text query matched case-insensitively against product name and description, paginated, newest first.
Endpoint	<code>GET /api/products/search?q=</code> PUBLIC

UC-7 — View a product's detail, gallery & reviews

Actor	Anyone (public)
Flow	Full product record with ordered image gallery (primary image flagged), average rating, review count, and the list of reviews.
Rules	Requesting an inactive product returns 404.
Endpoint	<code>GET /api/products/:id</code> , <code>GET /api/products/:id/reviews</code> PUBLIC

UC-8 — Browse categories & products in a category

Actor	Anyone (public)
Flow	Fetch the category tree (top-level categories with children); list paginated active products in a category by its slug.
Endpoint	<code>GET /api/categories</code> , <code>GET /api/categories/:slug/products</code> PUBLIC

C. Shopping Cart

UC-9 — View cart

Actor	Authenticated user
-------	--------------------

Flow	Returns the user's cart items with per-line totals, a running subtotal, total item count, price and stock for each, and the primary image.
Endpoint	<code>GET /api/cart</code> AUTH

UC-10 — Add item to cart

Actor	Authenticated user
Flow	Add a product + quantity. If the product is already in the cart, the quantities are merged (summed) rather than duplicated.
Rules	Inactive/unavailable products are rejected.
Endpoint	<code>POST /api/cart/items</code> AUTH

UC-11 — Update cart item quantity

Actor	Authenticated user (owner)
Flow	Set a new quantity. A quantity of zero or less deletes the line entirely.
Rules	A user can only touch their own cart items (ownership checked).
Endpoint	<code>PATCH /api/cart/items/:id</code> AUTH

UC-12 — Remove item / clear cart

Actor	Authenticated user (owner)
Flow	Remove a single line, or empty the whole cart.
Endpoint	<code>DELETE /api/cart/items/:id</code> , <code>DELETE /api/cart</code> AUTH

D. Addresses

UC-13 — Manage shipping addresses (list / add / update / delete)

Actor	Authenticated user (owner)
Flow	CRUD over the user's saved addresses. Marking an address as default automatically un-defaults the others.
Rules	Editing/deleting an address you don't own → access denied. Only one default per user.
Endpoint	<code>GET/POST /api/addresses</code> , <code>PUT/DELETE /api/addresses/:id</code> AUTH

E. Checkout & Orders

UC-14 — Place an order (checkout)

Actor	Authenticated user
Flow	From the cart, choose a shipping address and confirm. The server (in one transaction) verifies the cart isn't empty, re-checks stock for every line, decrements inventory, snapshots each line's unit price, computes subtotal + flat \$5.99 shipping = total, saves the order as PENDING , and empties the cart.
Rules	Empty cart → rejected. Any line exceeding available stock → InsufficientStockException and full rollback. Address must belong to the user.
Endpoint	POST /api/orders AUTH

UC-15 — View my order history

Actor	Authenticated user
Flow	Paginated list of the user's own orders, newest first, each with items, totals and status.
Endpoint	GET /api/orders AUTH

UC-16 — View a single order

Actor	Authenticated user (owner) or admin
Flow	Full order detail including line items, shipping address, and totals.
Rules	A regular user can only view their own order; an admin can view any order.
Endpoint	GET /api/orders/:id AUTH

F. Reviews

UC-17 — Read product reviews

Actor	Anyone (public)
Flow	List a product's reviews (rating, comment, reviewer name, date), newest first.
Endpoint	GET /api/products/:id/reviews PUBLIC

UC-18 — Write a review

Actor	Authenticated user
-------	--------------------

Flow	Submit a 1–5 star rating and an optional comment for a product.
Rules	Rating must be 1–5; a user may review a given product only once (duplicate → rejected).
Endpoint	<code>POST /api/products/:id/reviews</code> AUTH

G. Admin — Catalogue Management

UC-19 — Create a product

Actor	Admin
Flow	Provide name, description, price, stock, category and a list of image URLs; the first image becomes primary. Category must exist.
Endpoint	<code>POST /api/products</code> ADMIN

UC-20 — Edit a product

Actor	Admin
Flow	Update any field including active flag, category and the full image set (images are replaced when supplied).
Endpoint	<code>PUT /api/products/:id</code> ADMIN

UC-21 — Soft-delete (deactivate) a product

Actor	Admin
Flow	Sets <code>isActive=false</code> . The product disappears from all public listings and detail views but its historical order data is preserved.
Endpoint	<code>DELETE /api/products/:id</code> ADMIN

UC-22 — Create / edit categories

Actor	Admin
Flow	Create or update a category with a name, unique slug, and optional parent (enabling a category tree).
Endpoint	<code>POST /api/categories</code> , <code>PUT /api/categories/:id</code> ADMIN

H. Admin — Orders & Dashboard

UC-23 — View all orders

Actor	Admin
Flow	Paginated list of every order across all users, newest first.
Endpoint	<code>GET /api/orders/all</code> ADMIN

UC-24 — Update an order's status

Actor	Admin
Flow	Move an order between <code>PENDING</code> → <code>CONFIRMED</code> → <code>SHIPPED</code> → <code>DELIVERED</code> or <code>CANCELLED</code> via a dropdown.
Rules	Invalid status string → 400.
Endpoint	<code>PATCH /api/orders/:id/status</code> ADMIN

UC-25 — View the admin dashboard

Actor	Admin
Flow	Aggregate stats: total active products, total orders, total users, total revenue (excluding cancelled orders), plus a per-status order breakdown rendered as a Recharts bar chart.
Endpoint	<code>GET /api/admin/dashboard/stats</code> ADMIN

10. REST API Endpoint Catalogue

Base path `/api`. Access column: **PUB** public, **AUTH** any logged-in user, **ADM** admin only.

Method	Endpoint	Purpose	Access
POST	<code>/auth/register</code>	Create account, return JWT	PUB
POST	<code>/auth/login</code>	Authenticate, return JWT	PUB
GET	<code>/auth/me</code>	Current user profile	AUTH
GET	<code>/products</code>	List with filters/sort/paging	PUB
GET	<code>/products/:id</code>	Product detail	PUB
GET	<code>/products/search?q=</code>	Full-text search	PUB
POST	<code>/products</code>	Create product	ADM
PUT	<code>/products/:id</code>	Update product	ADM
DELETE	<code>/products/:id</code>	Soft-delete product	ADM
GET	<code>/categories</code>	Category tree	PUB
GET	<code>/categories/:slug/products</code>	Products in category	PUB
POST	<code>/categories</code>	Create category	ADM
PUT	<code>/categories/:id</code>	Update category	ADM
GET	<code>/cart</code>	View cart	AUTH
POST	<code>/cart/items</code>	Add item (merges qty)	AUTH
PATCH	<code>/cart/items/:id</code>	Update qty (0 removes)	AUTH
DELETE	<code>/cart/items/:id</code>	Remove item	AUTH
DELETE	<code>/cart</code>	Clear cart	AUTH
POST	<code>/orders</code>	Place order	AUTH
GET	<code>/orders</code>	My order history	AUTH
GET	<code>/orders/:id</code>	Order detail (owner/admin)	AUTH
GET	<code>/orders/all</code>	All orders	ADM
	<code>/orders/:id/status</code>		

PATCH		Update order status	ADM
GET	/addresses	List addresses	AUTH
POST	/addresses	Add address	AUTH
PUT	/addresses/:id	Update address	AUTH
DELETE	/addresses/:id	Delete address	AUTH
GET	/products/:id/reviews	List reviews	PUB
POST	/products/:id/reviews	Add review (1/user/product)	AUTH
GET	/admin/dashboard/stats	Dashboard aggregates	ADM

11. Frontend Structure & Routing

The React app is organised into pages, reusable components, two global context providers, custom hooks, and a per-domain Axios service layer. Global auth state lives in `AuthContext` (token, user, role, login/logout) and cart state in `CartContext` (items, subtotal, add/update/remove). Routes are guarded by `ProtectedRoute` (login required) and `AdminRoute` (admin required).

Route map

Path	Page	Guard
/	HomePage	PUBLIC
/products	ProductListPage (filters/search)	PUBLIC
/products/:id	ProductDetailPage (gallery, reviews)	PUBLIC
/login · /register	Login / Register	PUBLIC
/cart	CartPage	AUTH
/checkout	CheckoutPage	AUTH
/order-confirmation/:id	OrderConfirmationPage	AUTH
/orders	OrderHistoryPage	AUTH
/orders/:id	OrderDetailPage	AUTH
/admin	AdminDashboardPage (Recharts)	ADMIN
/admin/products	AdminProductsPage	ADMIN
/admin/orders	AdminOrdersPage	ADMIN

Axios interceptors

A request interceptor attaches the JWT to every call; a response interceptor catches `401` responses, clears the token, and redirects to `/login` — giving automatic session-expiry handling.

12. Running the Project & Seed Data

Prerequisites

Java 17+, Maven 3.8+, Node.js 18+, PostgreSQL 14+.

Steps

1. Create a PostgreSQL database named `ecommerce` (adjust credentials in `application.properties`).
2. Backend: `cd backend && mvn spring-boot:run` → starts on `:8080`. On first run Hibernate auto-creates the tables and `DataSeeder` seeds data.
3. Frontend: `cd frontend && npm install && npm run dev` → starts on `:5173`.

First-run gotcha

`spring.jpa.hibernate.ddl-auto` is set to `create`, which **drops and recreates all tables on every restart**. After the first successful run, switch it to `update` to keep your data between restarts. The seeder also skips itself if any users already exist.

Seeded accounts

Admins

Email	Password
<code>admin@store.com</code>	<code>admin123</code>
<code>superadmin@store.com</code>	<code>admin123</code>

Users

Email	Password
<code>user1@store.com</code>	<code>user123</code>
<code>user2@store.com</code>	<code>user123</code>
<code>user3@store.com</code>	<code>user123</code>

The seeder also creates the category tree and 15 sample products with images.

Note on secrets in this build

The bundled `application.properties` ships with a hard-coded JWT secret and DB password for local development. For any real deployment, move these to environment variables and rotate the JWT signing key.